

Segurança Informática

Licenciatura em Engenharia Informática

Relatório Experimental

Demonstração de Ataques XSS



UNIVERSIDADE DE ÉVORA

Miguel Pombeiro, 57829 | Miguel Rocha, 58501

Departamento de Informática
Universidade de Évora
Junho 2026

Índice

1	Objetivo	2
2	Metodologia	2
3	Resultados	3
3.1	DOM-Based XSS – Campo de Pesquisa	3
3.2	Reflected XSS – Consulta de Estado de Encomenda	3
3.3	Stored XSS – Registo de Utilizador e Roubo de Sessão	3
3.4	Stored XSS – Feedback e Reutilização da Sessão Roubada	4
4	Conclusão	5

1 Objetivo

Este trabalho teve como objetivo identificar, explorar e documentar alguns cenários de *Cross-Site Scripting* (XSS) na aplicação, intencionalmente vulnerável, OWASP Juice Shop, e mostrar o tema do trabalho teórico na prática. Sendo assim, cobriu-se as três categorias principais desta vulnerabilidade: *Dom-based XSS*, *Reflected XSS* e *Stored XSS* (persistente). Para além da exploração isolada de cada falha, procurou-se também demonstrar o impacto prático de um *Stored XSS* através de um cenário de ataque encadeado com o roubo da sessão de um utilizador administrador e a sua posterior reutilização para publicar conteúdo malicioso.

Numa fase inicial, pretendia-se complementar esta análise com a exploração da aplicação FoodApp, desenvolvida pelo Miguel Rocha no âmbito do estágio curricular. No entanto, tanto através de uma ferramenta automatizada de análise de vulnerabilidades (OWASP ZAP) como através de tentativas manuais de exploração, não foi possível identificar qualquer vulnerabilidade de XSS nesta aplicação. Este resultado é consistente com o facto de a FoodApp ter sido desenvolvida com recurso a *frameworks* modernas, que aplicam por defeito mecanismos de sanitização de conteúdo, dificultando significativamente a introdução deste tipo de falha. Por este motivo, optou-se por concentrar a análise exclusivamente no OWASP Juice Shop, uma aplicação desenhada deliberadamente para conter vulnerabilidades deste género.

2 Metodologia

A exploração de cada vulnerabilidade seguiu uma abordagem manual e iterativa, recorrendo principalmente à interação direta com a aplicação através do *browser*. O Burp Suite foi utilizado de forma seletiva, apenas nos casos em que era necessário modificar um pedido HTTP depois de este já ter passado pelas verificações do formulário, mas antes de chegar ao servidor – situação em que a interface da aplicação, por si só, não permite a exploração.

Em concreto, recorreu-se à funcionalidade *Intercept* do Burp Suite (com a configuração padrão da ferramenta) na submissão do formulário de registo de utilizador, para substituir o valor do campo de email por um *payload* malicioso que a validação do lado do cliente rejeitaria. Para além disso recorreu-se também às *dev-tools* do *browser* para substituir a *cookie* de sessão do pedido pela *cookie* de sessão do administrador, previamente obtida.

Nos restantes casos, a exploração foi realizada diretamente através da barra de endereços do *browser* ou do preenchimento de formulários, sem necessidade de capturar o tráfego, visto que nenhuma validação do lado do cliente impedia a submissão do *payload*.

3 Resultados

3.1 DOM-Based XSS – Campo de Pesquisa

Foi identificada uma vulnerabilidade de *DOM-based XSS* num mecanismo de confirmação de pesquisa. Embora a pesquisa de produtos comunique normalmente com o servidor, a mensagem de confirmação apresentada no cabeçalho da página de resultados é construída inteiramente no *browser*, a partir do texto presente na própria barra de endereços, sem ser feita qualquer verificação. Ao inserir um *payload* contendo uma *tag* HTML no próprio URL, o código foi interpretado e executado pelo *browser*.

Causa identificada: A principal causa desta vulnerabilidade é o excesso de confiança em dados controlados pelo utilizador, tratados como conteúdo seguro, que são apresentados diretamente, em vez de como texto simples.

Mitigação proposta: De forma a mitigar esta situação, o conteúdo proveniente do utilizador deve ser sempre apresentado como texto simples e sanitizado, permitindo que a própria tecnologia da página converta automaticamente quaisquer caracteres especiais, em vez de o tratar explicitamente como HTML de confiança.

3.2 Reflected XSS – Consulta de Estado de Encomenda

Foi identificada uma vulnerabilidade de *Reflected XSS* ao consultar o estado de uma encomenda. Ao submeter um ID inexistente, o servidor devolve esse mesmo ID na resposta, sem qualquer validação ou limpeza, sendo este posteriormente apresentado na página. Ao substituir o identificador por um *payload*, o conteúdo malicioso foi efetivamente enviado ao servidor e devolvido na resposta, sendo de seguida executado pelo *browser*.

Causa identificada: A causa desta ameaça foi a ausência de validação e de limpeza de um valor fornecido pelo utilizador, antes de este ser "refletido" numa resposta do servidor.

Mitigação proposta: Uma possível mitigação consiste na validação e sanitização no servidor de qualquer valor recebido antes de o incluir numa resposta (e.g. restringir o identificador a um conjunto seguro de caracteres). Do lado do cliente, o conteúdo recebido nunca deve ser tratado como HTML de confiança.

3.3 Stored XSS – Registo de Utilizador e Roubo de Sessão

Verificou-se que o formulário de registo de conta valida o formato do campo de email apenas do lado do cliente (*client-side*). Utilizando o Intercept do Burp Suite, foi possível capturar o pedido de criar conta após o preenchimento do formulário e substituir o valor do email por um *payload* malicioso antes de o reenviar ao servidor, contornando

assim a verificação que apenas existia na interface. O servidor aceitou o registo sem fazer qualquer validação própria.

Este *payload* foi desenhado para, em vez de apresentar apenas um alerta, ler e expor o valor da *cookie* de sessão presente no *browser*. Ao autenticar como administrador e aceder à página de administração – onde a lista de utilizadores, incluindo o email malicioso, é apresentada – o código foi executado no contexto da sessão do administrador, permitindo a obtenção do valor da sua *cookie* de sessão.

Causa identificada: A principal causa foi a falta de validação do lado do servidor, dependente exclusivamente de uma verificação do lado do cliente facilmente contornável.

Mitigação proposta: Qualquer validação de dados deve ser sempre garantida do lado do servidor, independentemente da forma como o pedido é recebido. Adicionalmente, os valores de sessão devem ser protegidos contra leitura por código a correr na página, e o seu tempo de vida deve ser limitado.

3.4 Stored XSS – Feedback e Reutilização da Sessão Roubada

Foi identificada uma segunda vulnerabilidade de XSS persistente no formulário de comentários de *feedback*. A limpeza de conteúdo aplicada a este campo revelou-se insuficiente, inicialmente foram testados *payloads* simples ou com um único nível de "disfarce" mas que não foram incluídos no vídeo. A tentativa que conseguiu entregar o *payload* foi construída fechando uma *tag script* imediatamente após a sua abertura, sobrevivendo, assim, ao processo de limpeza, mantendo o código malicioso intacto.

De forma a demonstrar o impacto do roubo de sessão obtido no cenário anterior, a *cookie* de sessão do administrador foi reutilizada – através da sua substituição direta nas ferramentas de desenvolvedor do *browser* – para submeter este comentário autenticado como administrador, em vez de como utilizador comum. O comentário malicioso ficou assim associado à conta do administrador, sem que a respetiva *password* tivesse sido conhecida ou utilizada em qualquer momento. Ao recarregar a página de administração e a página pública "about us", o *payload* foi executado automaticamente perante qualquer visitante.

Causa identificada: A causa identificada foi uma má filtragem do conteúdo recebido, eram apenas feita uma limpeza de conteúdo numa única passagem, o que foi contornável através de *payloads* com camadas de "disfarce" aninhadas. Foi também combinada a possibilidade de reutilização de uma sessão comprometida sem qualquer mecanismo adicional de verificação, tendo sido, portanto, aplicadas duas vulnerabilidades numa única demonstração.

Mitigação proposta: A limpeza do conteúdo deve ser aplicada de forma recursiva, repetindo a verificação sobre o próprio resultado até não restar qualquer conteúdo estranho que deva ser removido. A proteção do valor de sessão, descrita na secção anterior, é

igualmente determinante para impedir este cenário de reutilização.

4 Conclusão

A exploração manual das quatro vulnerabilidades confirmou que, embora a causa imediata de cada falha seja distinta – confiança indevida em dados de URL, reflexão de dados sem validação, ou limpeza de conteúdo insuficiente – o problema vem da confiança em dados fornecidos pelo utilizador sem os tratar de forma adequada antes de os apresentar ou de os armazenar.

O cenário encadeado dos últimos dois casos permitiu ainda demonstrar que o impacto de um XSS persistente não se limita à execução de código no *browser* da vítima, porque um valor de sessão exposto pode ser reutilizado para agir em nome dessa vítima perante o servidor, sem qualquer conhecimento da sua *password*. Este resultado reforça a importância da validação e da limpeza de dados deve ocorrer sempre do lado do servidor, de forma completa e também com verificações *client-side*. Para além disso, os valores de sessão devem ser protegidos contra leitura por código em execução na página, de forma a limitar o impacto de uma eventual vulnerabilidade XSS já existente.